

Creating a React App with Login, Dashboard, and Authentication

Here's a step-by-step guide to create a simple React application with authentication, login, and dashboard pages using routing in VS Code.

Step 1: Set Up the Project

1. Open VS Code
2. Open the terminal (Ctrl+` or Terminal > New Terminal)
3. Run these commands:

```
npx create-react-app react-auth-app
cd react-auth-app
npm install react-router-dom
```

Step 2: Create the Basic File Structure

Create these files in your `src` folder:

```
src/
├── components/
│   ├── PrivateRoute.js
│   └── Navbar.js
├── pages/
│   ├── Login.js
│   ├── Dashboard.js
│   └── Home.js
├── App.js
└── index.js
```

Step 3: Implement the Components

1. PrivateRoute.js (for protected routes)

```
import { Navigate, Outlet } from 'react-router-dom';

const PrivateRoute = () => {
  const isAuthenticated = localStorage.getItem('token'); // Simple auth check
  return isAuthenticated ? <Outlet /> : <Navigate to="/login" />;
}
```

```
};

export default PrivateRoute;
```

2. Navbar.js

```
import { Link, useNavigate } from 'react-router-dom';

const Navbar = () => {
  const navigate = useNavigate();
  const isLoggedIn = localStorage.getItem('token');

  const handleLogout = () => {
    localStorage.removeItem('token');
    navigate('/login');
  };

  return (
    <nav>
      <Link to="/">Home</Link>
      {isLoggedIn ? (
        <>
          <Link to="/dashboard">Dashboard</Link>
          <button onClick={handleLogout}>Logout</button>
        </>
      ) : (
        <Link to="/login">Login</Link>
      )}
    </nav>
  );
};

export default Navbar;
```

3. Login.js

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';

const Login = () => {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const navigate = useNavigate();

  const handleSubmit = (e) => {
    e.preventDefault();
```

```

    // Simple authentication - in real app, you'd call an API
    if (username === 'admin' && password === 'password') {
      localStorage.setItem('token', 'dummy-token');
      navigate('/dashboard');
    } else {
      alert('Invalid credentials');
    }
  };

  return (
    <div>
      <h2>Login</h2>
      <form onSubmit={handleSubmit}>
        <div>
          <label>Username:</label>
          <input
            type="text"
            value={username}
            onChange={ (e) => setUsername(e.target.value) }
          />
        </div>
        <div>
          <label>Password:</label>
          <input
            type="password"
            value={password}
            onChange={ (e) => setPassword(e.target.value) }
          />
        </div>
        <button type="submit">Login</button>
      </form>
    </div>
  );
};

export default Login;

```

4. Dashboard.js

```

const Dashboard = () => {
  return (
    <div>
      <h2>Dashboard</h2>
      <p>Welcome to your dashboard!</p>
    </div>
  );
};

```

```
};

export default Dashboard;
```

5. Home.js

```
const Home = () => {
  return (
    <div>
      <h2>Home Page</h2>
      <p>Welcome to our website!</p>
    </div>
  );
};

export default Home;
```

Step 4: Update App.js

```
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
import Home from './pages/Home';
import Login from './pages/Login';
import Dashboard from './pages/Dashboard';
import PrivateRoute from './components/PrivateRoute';

function App() {
  return (
    <Router>
      <Navbar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/login" element={<Login />} />
        <Route element={<PrivateRoute />>
          <Route path="/dashboard" element={<Dashboard />} />
        </Route>
      </Routes>
    </Router>
  );
}

export default App;
```

Step 5: Update index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Step 6: Add Basic Styling (optional)

Add this to `src/index.css`:

```
nav {
  background: #f0f0f0;
  padding: 1rem;
  margin-bottom: 1rem;
}

nav a {
  margin-right: 1rem;
  text-decoration: none;
  color: #333;
}

nav a:hover {
  text-decoration: underline;
}

form div {
  margin-bottom: 1rem;
}

label {
  display: inline-block;
  width: 80px;
}
```

Step 7: Run the Application

In the terminal, run:

```
npm start
```

Your app should now be running on `http://localhost:3000` with:

- A home page accessible to everyone
- A login page (try username: "admin", password: "password")
- A dashboard page that's protected and only accessible after login
- Navigation that changes based on authentication status